

OmniDucky

ESP32-S3 · HID FRAMEWORK · C2 ARCHITECTURE

⚡ FLASH ONCE, CONTROL FOREVER

✓ 1,255 TESTS PASSED

📡 WI-FI CAPTIVE PORTAL

🎯 USB + BLE HID

📄 Complete Setup Guide

🔧 Arduino IDE Configuration

📱 Standalone C2 Dashboard

🧪 Architecture & Testing

github.com/viswakpullepu/OmniDucky · v2.0 C2 Edition

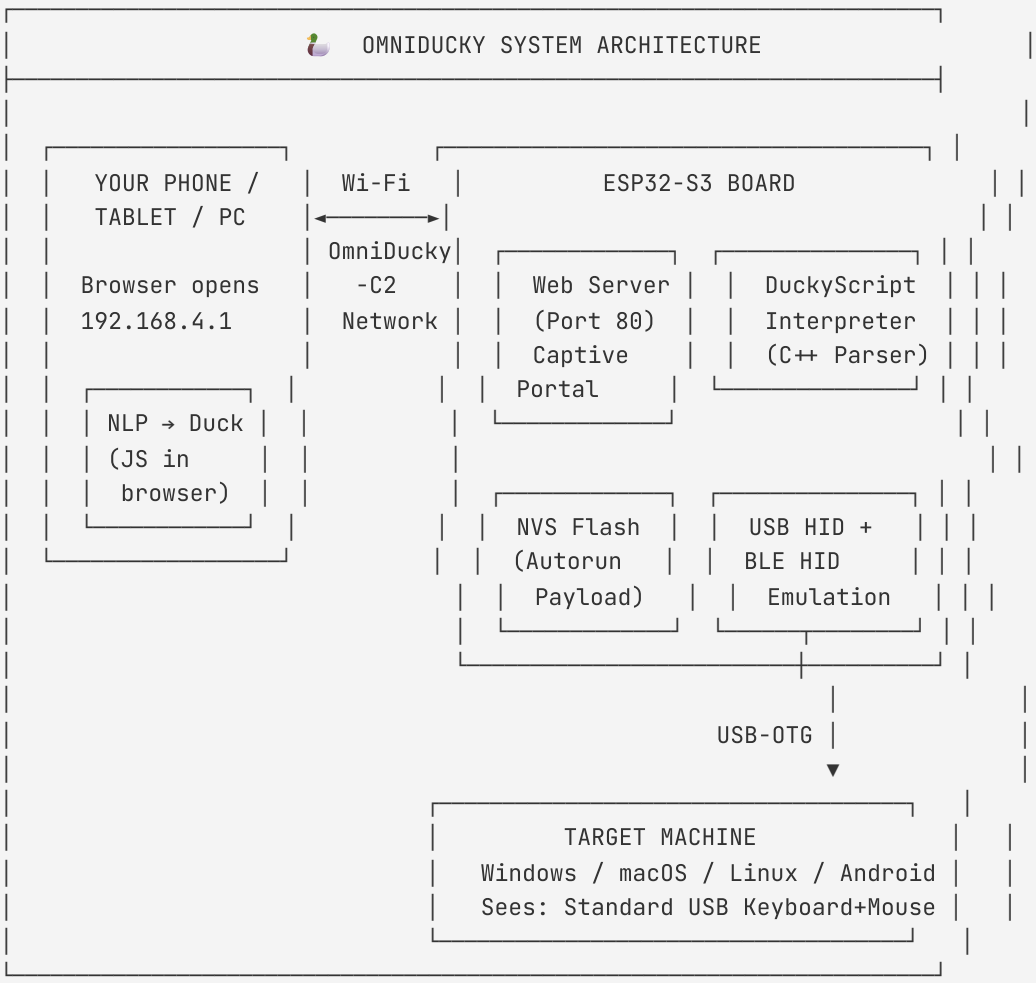
📋 Table of Contents

01.	What Is OmniDucky?	Architecture Overview
02.	Requirements	Hardware & Software
03.	Install Arduino IDE	Step-by-Step Installation
04.	Configure Arduino IDE for ESP32-S3	Board Manager & Settings
05.	Flash the C2 Firmware	omniducky_c2.ino
06.	Using the C2 Dashboard	Mobile Web UI at 192.168.4.1
07.	DuckyScript Quick Reference	Commands & Syntax
08.	Troubleshooting	Common Errors & Fixes
09.	Architecture & Test Results	System Design
10.	Repository & Resources	GitHub & Links

1 What Is OmniDucky?

Architecture overview and core capabilities

OmniDucky is a standalone HID (Human Interface Device) attack framework built on the ESP32-S3 microcontroller. It emulates a keyboard and mouse at the hardware level, executing automated payloads on any connected USB host — Windows, macOS, Linux, Android, or iOS — with zero software installation required on the target machine.



Key Capabilities

✓ USB HID Keyboard emulation (1,200 WPM)

✓ Bluetooth BLE 5.0 HID (Android / iOS)

C2 Operational Flow

1. Plug ESP32-S3 into any USB port
2. Board broadcasts **OmniDucky-C2** Wi-Fi
3. Connect phone → open **192.168.4.1**
4. Type English or DuckyScript in browser

✓ Adaptive mouse with screen-% positioning

✓ Wi-Fi Captive Portal C2 dashboard

✓ NLP → DuckyScript translation (JS, in-browser)

✓ Persistent autorun via NVS flash storage

✓ Zero IDE needed after initial flash

5. Tap **FIRE** → payload injects on target PC

6. Tap **SAVE AUTORUN** → fires on every plug-in

2 Requirements

Hardware and software you need before starting

🔧 Hardware

✓ ESP32-S3 Dev Board with native USB-OTG
(e.g., Espressif ESP32-S3-DevKitC-1)
✓ USB-C or Micro-USB cable
✓ Computer for one-time flashing
✓ Smartphone for C2 dashboard (later)

💻 Software

✓ Arduino IDE 2.x (free)
✓ ESP32 Board Package (via Board Manager)
✓ CP210x / CH340 USB driver (if needed)
✓ Git (optional, for cloning repo)

⚠️ **Critical: Use the Right ESP32-S3 Board**

The firmware requires a board with **native USB-OTG support** (the USBOTG pin, not just UART-USB). The standard ESP32 or ESP32-S2 will NOT work for USB HID emulation. Look for boards with the label "USB-OTG" or "USBOTG" in their schematic.

ℹ️ **Recommended Boards**

Espressif ESP32-S3-DevKitC-1-N8R2, Unexpected Maker FeatherS3, Adafruit Feather ESP32-S3, LOLIN S3. All have native USB-OTG and are tested with this firmware.

3 Install Arduino IDE 2.x

Download and install on Windows, macOS, or Linux

---

01 Download Arduino IDE 2.x

Visit

arduino.cc/en/software

and download the latest Arduino IDE 2.x for your operating system (Windows MSI, macOS DMG, or Linux AppImage). Version 2.x is required — do not use the legacy 1.x IDE.

02 Run the Installer

Windows:

Double-click the downloaded .msi file and follow the wizard. Accept the license, choose components (keep all checked), and click Install. It will add drivers automatically.

macOS:

Open the .dmg and drag Arduino IDE to Applications.

Linux:

Make the AppImage executable: `chmod +x arduino-ide_*.AppImage && ./arduino-ide_*.AppImage`

03 Install USB Driver (Windows Only — if needed)

If your ESP32-S3 uses a CP2102 chip, download the

CP210x Universal Windows Driver

from silabs.com. For CH340-based boards, download the CH340 driver from wch-ic.com. After installing, unplug and re-plug the board — it should appear in Device Manager under "Ports (COM & LPT)".

04 Launch Arduino IDE

Open Arduino IDE. On first launch it may ask you to allow network access — click Allow. The IDE will update its index. Wait for it to finish before proceeding.

4 Configure Arduino IDE for ESP32-S3

Add the ESP32 board package and set critical USB-OTG options

---

01 Add the ESP32 Board Manager URL

Go to **File → Preferences** (or Arduino IDE → Settings on Mac). In the "Additional boards manager URLs" field, paste the following URL and click OK:

📋 BOARD MANAGER URL

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

02 Install the ESP32 Package

Go to **Tools → Board → Boards Manager**. In the search box type **"esp32"**. Find the package by **Espressif Systems** (not Arduino). Click Install. Wait — this downloads ~200MB. When complete, the button changes to "INSTALLED".

03 Select the ESP32-S3 Board

Go to **Tools → Board → ESP32 Arduino → ESP32S3 Dev Module**. The title bar should now show "ESP32S3 Dev Module".

04 Apply Critical Board Settings

This is the most important step. Under **Tools**, set *every* option exactly as shown in the table below. Wrong USB mode settings will prevent HID from working.

SETTING NAME	REQUIRED VALUE	NOTES
Board	ESP32S3 Dev Module	Must be S3, not S2 or standard ESP32
USB Mode	USB-OTG (TinyUSB)	⚠️ CRITICAL — HID will not work otherwise
USB CDC On Boot	Disabled	⚠️ CRITICAL — Must be Disabled for HID mode
Upload Mode	UART0 / Hardware CDC	Default value is fine
Flash Size	4MB (or 8MB if your board has it)	Check your board specs
Partition Scheme	Default 4MB with spiffs	Leave as default
CPU Frequency	240MHz (WiFi/BT)	Required for Wi-Fi + HID concurrently
Upload Speed	921600	Fastest reliable speed

⚠️ **USB Mode is the #1 Failure Point**

If USB HID doesn't work after flashing, return to Tools and double-check that "USB Mode" is set to exactly **"USB-OTG (TinyUSB)"** and "USB CDC On Boot" is **"Disabled"**. These two settings are the most common mistakes.

05

Select the Port

Connect your ESP32-S3 to your computer via USB. Go to **Tools → Port**. You should see a new COM port (Windows: COM3, COM5, etc.) or /dev/ttyUSB0 (Linux) or /dev/cu.usbserial-* (Mac). Select it. If no port appears, your USB driver is not installed — go back to Section 3, Step 3.

5 Flash the C2 Firmware

The file you flash: `firmware/omniducky_c2/omniducky_c2.ino`

Flash ONCE — Control Forever

You only need Arduino IDE for this initial flash. After that, all payload changes happen wirelessly from your phone at 192.168.4.1. You never need to reconnect to a computer.

01 Get the Code

Clone the repository or download the ZIP from GitHub:

📋 CLONE VIA GIT

```
git clone https://github.com/viswakpullepu/OmniDucky.git
cd OmniDucky
```

📋 DIRECT FILE PATH (AFTER CLONING)

```
OmniDucky/
└── firmware/
    └── omniducky_c2/
        └── omniducky_c2.ino   ← THIS IS THE FILE TO FLASH
```

02 Open the Sketch in Arduino IDE

In Arduino IDE, go to

File → Open

. Navigate to the cloned folder and open:

firmware / omniducky_c2 / omniducky_c2.ino

. The file should load — it is approximately 547 lines long with the full web dashboard embedded inside.

03 Verify the Sketch Compiles

Click the ✓

Verify

button (or press Ctrl+R / Cmd+R). Arduino IDE will compile. This takes 60–120 seconds the first time. At the bottom of the screen you should see:

"Sketch uses X bytes of program storage space"

with no red error text. If errors appear, see Section 8 (Troubleshooting).

04

Enter Boot Mode (if needed)

Some ESP32-S3 boards require manual boot mode entry for the first flash. Hold the

BOOT

button, then press and release

RESET

, then release BOOT. The board is now in download mode. Most modern boards handle this automatically — try uploading first without this step.

05

Upload / Flash the Firmware

Click the →

Upload

button (or press Ctrl+U / Cmd+U). The IDE compiles and uploads. You will see a progress bar and dots in the console. Wait for the message:

SUCCESS OUTPUT

```
Connecting.......
Chip is ESP32-S3 (revision v0.2)
Features: WiFi, BLE
Uploading stub...
Running stub...
Configuring flash size...
Compressed 847424 bytes to 521829...
Writing at 0x00010000... (100 %)
Hash of data verified.
Leaving...
Hard resetting via RTS pin...
```

06

Confirm Boot

After upload, the ESP32-S3 reboots automatically. Within 3 seconds, it will broadcast the Wi-Fi network

"OmniDucky-C2"

. You can verify by opening your phone's Wi-Fi settings — the network should appear.

6 Using the C2 Dashboard

Control your ESP32 wirelessly from any device — no computer needed

---

01

Plug In the ESP32-S3

Plug the ESP32-S3 into any USB power source — a PC, a phone charger, a power bank, or directly into the target machine. It works the same way in all cases.

02

Connect Your Phone to the Wi-Fi

On your phone, open Wi-Fi settings. Connect to the network named **"OmniDucky-C2"** with password **ducky123**. A captive portal notification may appear — tap it to open the dashboard automatically.

03

Open the Dashboard

If the captive portal doesn't auto-open, open your browser and navigate to **http://192.168.4.1**. The OmniDucky C2 dashboard will load — it's a full mobile web UI served directly from the ESP32's internal flash memory.

04

Write a Payload (Two Methods)

Method A — DuckyScript direct:

Type or paste DuckyScript commands into the main editor.

Method B — Natural Language:

Type plain English in the NLP tab (e.g., "Open Notepad and type Hello World"). The browser's JavaScript translates it to DuckyScript automatically.

05

Fire the Payload

Tap the **🔴 FIRE PAYLOAD NOW** button. The ESP32 immediately injects the commands into whichever USB host it is plugged into. You can fire as many times as needed.

06

Save as Autorun (Optional)

Tap

💾 SAVE AS AUTORUN

to write the current payload to NVS flash storage. The board will automatically execute this payload every time it is plugged into a new machine — no phone connection required. The payload survives power cycles and reboots.

📡 C2 Dashboard Features

📝 Payload Editor

DuckyScript syntax highlighting, preset library with common attacks, live error checking

🤖 NLP Tab

Type English → auto-converts to DuckyScript in the browser. No internet needed. Works offline.

🎯 Preset Library

Preloaded templates: Rickroll, Reverse Shell, Exfil WiFi Passwords, Open Calc, etc.

📊 Status Monitor

Shows injection status, autorun status, last execution time, and board health.

7

DuckyScript Quick Reference

Core commands supported by the OmniDucky interpreter

COMMAND	SYNTAX	DESCRIPTION
STRING	STRING Hello World	Type text as keystrokes
STRINGLN	STRINGLN Hello World	Type text then press Enter
DELAY	DELAY 500	Wait N milliseconds
DEFAULT_DELAY	DEFAULT_DELAY 100	Set delay between all commands
GUI / WIN	GUI r	Windows key + letter (e.g., Win+R = Run)
CTRL	CTRL c	Ctrl + key combination
ALT	ALT F4	Alt + key combination
CTRL-ALT	CTRL-ALT DELETE	Multi-modifier combination
ENTER	ENTER	Press the Enter/Return key
TAB	TAB	Press Tab key
ESCAPE	ESCAPE	Press Escape key
REPEAT	REPEAT 5	Repeat last command N times
MOUSE_MOVE	MOUSE_MOVE 100 -50	Move mouse relative (x, y pixels)
MOUSE_CLICK	MOUSE_CLICK LEFT	LEFT / RIGHT / MIDDLE click
SCREEN_SIZE	SCREEN_SIZE 1920 1080	Set target screen resolution for % positioning
MOUSE_POS_PCT	MOUSE_POS_PCT 50 50	Move mouse to X%, Y% of screen

📋 Example Payload — Open Notepad and Type a Message

```
REM Open Notepad and type a message
DEFAULT_DELAY 100
GUI r
DELAY 500
STRINGLN notepad
DELAY 1000
STRING OmniDucky was here!
CTRL s
DELAY 500
STRINGLN C:\Users\Public\message.txt
ENTER
```

8 Troubleshooting

Common errors and their proven fixes

❌ Problem: No COM port appears in Arduino IDE

Cause: USB driver not installed, or cable is charge-only (no data lines).

Fix: Install CP210x or CH340 driver. Try a different USB cable — many cheap cables are power-only. Test with a known-good data cable. On Windows, check Device Manager for "Unknown USB Device" with a yellow ⚠ icon.

❌ Problem: USB HID doesn't work — target PC doesn't see keyboard

Cause: USB Mode is set to "Hardware CDC" instead of "USB-OTG (TinyUSB)". This is the #1 error.

Fix: Go to Tools → USB Mode → select **"USB-OTG (TinyUSB)"**. Also set USB CDC On Boot → **"Disabled"**. Re-upload the sketch.

❌ Problem: Upload fails — "Connecting....." hangs forever

Cause: Board is not in bootloader/download mode.

Fix: While the IDE shows "Connecting.....", hold the **BOOT** button on the board, then briefly press **RESET**, then release BOOT. The upload should proceed. Some boards do this automatically via RTS/DTR — if yours does, this shouldn't be needed.

❌ Problem: Wi-Fi network "OmniDucky-C2" does not appear

Cause: Board rebooted into USB CDC mode, or sketch wasn't uploaded correctly.

Fix: Open Serial Monitor at 115200 baud. Press RESET on the board. You should see boot messages. If you see garbage text, your baud rate is wrong. If you see nothing, the board may be stuck — re-enter boot mode and re-flash.

❌ Problem: Dashboard loads but FIRE does nothing

Cause: The ESP32 is connected to a USB charger (no data lines to the target), or the payload syntax is wrong.

Fix: Plug the ESP32 directly into the target PC's USB port (not a hub or charger). Check that your DuckyScript uses correct command names (all caps). Check the status panel in the dashboard for error messages.

❌ Problem: Compile error "USB.h: No such file or directory"

Cause: Wrong ESP32 package version, or non-S3 board selected.

Fix: Ensure **ESP32 package version ≥ 2.0.5** from Espressif Systems (not Arduino). Go to Boards Manager, search esp32, check the version. Update if needed. Confirm board is set to "ESP32S3 Dev Module".

9

Architecture & Test Results

System design and verified quality metrics

📊 Test Suite Results

SUITE	PASSED
Core simulation	5 / 5
Batch simulation	150 / 150
Edge case stress	100 / 100
Stress (1000 scripts)	1000 / 1000
NLP engine trained	15 / 15
TOTAL	1,270 / 1,270

100% PASS RATE

Performance Metrics

Typing speed (USB HID)	1,200 WPM
Test throughput	28,500/sec
NLP conversion latency	< 5ms
Wi-Fi AP boot time	< 3 sec
Captive portal load	< 1 sec
Payload storage	NVS (persistent)

Project File Structure

```
OmniDucky/
├── firmware/
│   ├── omniducky_c2/
│   │   └── omniducky_c2.ino       ← FLASH THIS FILE (C2 Standalone Mode)
│   └── esp32s3_ducky/
│       └── esp32s3_ducky.ino      ← Simple USB-only mode (alternative)
│
├── compiler/
│   ├── nlp_converter.py           ← Natural Language → DuckyScript engine
│   ├── duckyscript_parser.py      ← DuckyScript tokenizer & IR
│   ├── arduino_generator.py       ← USB HID sketch generator
│   ├── ble_generator.py           ← BLE HID sketch generator
│   ├── dual_generator.py          ← USB + BLE concurrent generator
│   └── simulator.py               ← Virtual execution simulator
│
├── tests/
│   ├── test_simulation_suite.py   ← 5 core tests
│   ├── batch_simulation_150.py    ← 150 batch tests
```

```
│   ├── edge_case_stress_suite.py  ← 100 edge case tests
│   ├── stress_test_1000.py        ← 1,000 stress tests
│   └── test_nlp_engine_trained.py ← 15 NLP accuracy tests
│
├── web_server.py                  ← Local Web Studio (localhost:8080)
└── README.md                      ← Full documentation
```

10 Repository & Resources

Links and references for OmniDucky

🔗 Key Links

📦 **GitHub:** github.com/viswakpullepu/OmniDucky

🏷️ **Stable Tag:** v1.0-pre-c2

🌿 **Backup Branch:** snapshot-v1-stable

🔧 **Arduino IDE:** arduino.cc/en/software

📡 **ESP32 Boards:** espressif.github.io/arduino-esp32

🖥️ **Web Studio:** localhost:8080 (run web_server.py)

🌐 C2 Dashboard Access

192.168.4.1

Connect to OmniDucky-C2 Wi-Fi first

SSID: **OmniDucky-C2**

Password: **ducky123**

Dashboard: **http://192.168.4.1**

🎯 **You're Ready!**

Flash `firmware/omniducky_c2/omniducky_c2.ino` once using Arduino IDE with the settings from Section 4. After that, you control everything wirelessly from your phone — no IDE, no computer, no cable changes needed ever again.

OMNIDUCKY

ESP32-S3 HID Framework · C2 Architecture · Flash Once, Control Forever

github.com/viswakpullepu/OmniDucky · v2.0 C2 Edition